



가천대학교 스마트시티학과 – 가상공간과 메타버스

Lecture 11 – Agent-based City Modeling Lab

Chris Ohk

utilForever@gmail.com

- Lecture 10의 행동 규칙을 기존 도시 프로그램에 적용한다.
- 함수 7개를 작성해 만족도 / 이주 / 기업 이전 / 이웃 비율을 구현한다.
 - 작성 → 단계 검사 → 화면 확인 → 조건을 바꿔 실험

- 압축을 푼 스켈레톤 폴더에서 프로그램을 실행한다.
 - Windows: `py -m tkinter → py app.py`
 - macOS / Linux: `python3 -m tkinter → python3 app.py`
- `agent_rules.py`를 저장한 뒤 앱을 닫고 다시 실행한다.

- 제공된 코드에서 TODO 주석 부분을 채운다.

파일	역할	수업 중 수정
agent_rules.py	행동 규칙 함수 7개	직접 작성
abm_support.py	후보 생성 / 배정 / 월별 갱신	제공
app.py / abm_gui.py	실행 / 행위자 화면	제공
neighborhood.py	이웃 모형의 진행	제공
checks_abm.py	단계별 자동 검사	실행

첫 화면 살펴보기

- 시민 / 가구 / 기업 탭에서 주거지와 직장, 현재 만족도를 확인한다.

행위자 상태

0

R

A

2 / 4

1

R

C

0 / 4

2

R

B

0 / 4

3

R

D

1 / 4

물

1

물

2

3

C

Y

0 / 8

P

공원

C

X

2 / 8

A~D: 주택 / X~Y: 사업장 / P: 공원

시민	가구	주택	기업	통근(분)
P1	H1	A	F1	45
P2	H1	A	없음	—
P3	H2	D	F1	15

주거지는 소속 가구에서 조회합니다.
통근 시간은 현재 도로와 사업장 위치로 계산합니다.

1개월 진행

6개월 진행

월별 CSV

가구

가구	주택	만족도	저축	연속 불만(월)	대기(월)
H1	A	40.5	150	0	0
H2	D	72.25	100	0	0

조사할 가구

H1

순서대로 최대 3곳 / 개선 폭 10점 이상

후보 주택	만족도	임대료	입주 가능
B	62.33	100	가능
C	43.33	130	불가
D	70	90	불가

기업

기업	사업장	월 이익	이전 자금	고용 시민
F1	X	150	700	P1 / P3

분기마다 이전 검토 / 앞으로의 6개월 이익 비교 / 시민의 고용 관계 유지

- 시민은 가구와 기업의 ID를 통해 주거지와 직장을 참조한다.

자료	주요 상태	연결된 자료
citizens	household_id / employer_id	가구 / 기업
households	home_id / savings / cooldown	주택
firms	site_id / cash / profit	사업장
houses / sites	occupant	입주한 가구 / 기업

이번 실습의 도시

- 시민 3명이 두 가구를 이루며, 취업 시민 2명은 기업 F1에 다닌다.

대상	초기 위치 / 구성	주요 조건
H1	주택 A / P1과 P2	월소득 200 / 저축 150
H2	주택 D / P3	월소득 240 / 저축 100
빈집	B / C	월세 100 / 130
F1	사업장 X / 직원 P1과 P3	월 이익 150 / 자금 700

- `satisfaction` 함수를 작성한다.
 - 입력 : 통근 시간 / 월세 / 월소득 / 환경 점수
 - 통근과 주거비 점수를 구한 뒤 환경 점수와 가중합을 계산한다.
 - 반환값 : 0~100점의 만족도

- H1의 만족도는 주택 A에서 40.5점, 주택 B에서 약 62.3점이다.

항목	계산 방법	주택 A	주택 B
통근 점수 C	$100 \times \max(0, 1 - \text{시간}/60)$	25.0	약 66.7
주거비 점수 R	$100 \times \max(0, 1 - \text{월세}/\text{소득})$	60.0	50.0
환경 점수 E	주택에 저장된 값	50.0	70.0
만족도 S	$0.5C + 0.3R + 0.2E$	40.5	약 62.3

- `agent_rules.py`의 TODO 1 부분을 완성해 보자.
 - `python checks_abm.py --stage 1`
 - `satisfaction(45, 80, 200, 50)`의 결과 : 40.5
- 통근 경로가 없거나 월소득이 0이면 어떻게 계산할까?

- 통근과 주거비 점수의 최솟값을 0으로 제한한 뒤 가중합을 반환한다.



```
def satisfaction(commute_min, rent, income, environment):  
    if commute_min is None:  
        commute_score = 0  
    else:  
        commute_score = 100 * max(0, 1 - commute_min / 60)  
  
    rent_score = 0 if income == 0 else 100 * max(0, 1 - rent / income)  
    return 0.5 * commute_score + 0.3 * rent_score + 0.2 * environment
```

실습 2 : 이주 검토

- `should_move` 함수를 작성한다.
 - 입력 : 가구의 연속 불만족 기간 / 재이주 대기 기간 / 저축
 - 불만족 2개월 이상, 대기 0개월, 이사 비용 이상의 저축이 필요하다.
 - 반환값 : 이주를 검토할 수 있으면 `True`, 아니면 `False`

이주를 검토하는 조건

- 세 조건을 모두 충족할 때만 True를 반환한다.

연속 불만족	재이주 대기	저축 / 비용	반환값
1개월	0개월	150 / 30	False
2개월	0개월	30 / 30	True
2개월	1개월	150 / 30	False
2개월	0개월	29 / 30	False

- `agent_rules.py`의 TODO 2 부분을 완성해 보자.
 - `python checks_abm.py --stage 2`
 - 불만족 2개월 / 대기 0개월 / 저축 30 / 비용 30 : True
- 세 조건 중 하나라도 충족하지 못하면 어떤 값을 반환할까?

- 세 조건을 and로 연결해 이주 검토 여부를 반환한다.



```
def should_move(household):  
    return (household["dissatisfied_months"] >= 2  
            and household["cooldown"] == 0  
            and household["savings"] >= household["move_cost"])
```

실습 3 : 주택 선택

- `choose_home` 함수를 작성한다.
 - 입력 : 가구 / 후보 목록 / 현재 만족도
 - 최대 3개 후보를 순서대로 살펴본다.
 - 입주할 수 있고 만족도가 10점 이상 높은 첫 후보를 선택한다.
 - 반환값 : 선택한 후보의 디셔너리 / 선택하지 못하면 `None`

후보를 순서대로 살펴보기

- 현재 40.5점이면 첫 번째 후보 P를 선택하고 탐색을 끝낸다.

조사 순서	후보	점수	입주 가능	처리
1	P	55.0	True	10점 이상 개선 / 선택
2	Q	80.0	True	선택 후에는 조사하지 않음
3	R	70.0	True	선택 후에는 조사하지 않음

- `agent_rules.py`의 TODO 3 부분을 완성해 보자.
 - `python checks_abm.py --stage 3`
 - 현재 40.5점 / P 55점 / Q 80점 / R 70점 : P 선택
- 네 번째 후보만 조건을 충족한다면 어떤 결과가 나올까?

- 앞의 세 후보 중 조건을 충족하는 첫 후보를 반환한다.

```
def choose_home(household, candidates, current_score):  
    for candidate in candidates[:3]:  
        if candidate["feasible"] and candidate["score"] >= current_score + 10:  
            return candidate  
  
    return None
```

실습 4 : 가구 이주

- `move_household` 함수를 작성한다.
 - 입력 : 행위자 상태 / 가구 ID / 승인된 주택 ID
 - 이전 주택을 비우고 새 주택과 가구의 입주 정보를 맞춘다.
 - 이사 비용을 한 번 차감하고, 불만족 기간을 0 / 대기를 3으로 바꾼다.
 - 반환값 없이 전달받은 상태를 갱신한다.

가구 이주 전후의 상태

- H1이 A에서 B로 이사하면 입주 관계와 저축이 함께 바뀐다.

상태	이주 전	이주 후
H1의 주택	A	B
주택 A / B	H1 거주 / 빈집	빈집 / H1 거주
저축	150만원	120만원
불만족 / 대기	2개월 / 0개월	0개월 / 3개월

- `agent_rules.py`의 TODO 4 부분을 완성해 보자.
 - `python checks_abm.py --stage 4`
 - H1의 주택 B / 저축 120 / 불만족 0 / 대기 3
- 가구의 주택 ID만 바꾸면 기존 주택에는 어떤 오류가 남을까?

- 두 주택의 입주 정보와 가구의 주택 ID를 함께 갱신한다.



```
def move_household(state, household_id, new_home_id):  
    household = state["households"][household_id]  
    old_home_id = household["home_id"]  
  
    state["houses"][old_home_id]["occupant"] = None  
    state["houses"][new_home_id]["occupant"] = household_id  
  
    household["home_id"] = new_home_id  
    household["savings"] -= household["move_cost"]  
    household["dissatisfied_months"] = 0  
    household["cooldown"] = 3
```

실습 5 : 기업 이전 판단

- `should_move_firm` 함수를 작성한다.
 - 입력: 기업의 현재 상태 / 후보 사업장의 월 이익
 - 향후 6개월의 이익 증가분에서 이전 비용을 뺀다.
 - 순편익이 양수이고 현금이 충분하면 `True`를 반환한다.

이익 증가분과 이전 비용

- 6개월의 이익 증가분은 660만원이고, 이전 비용을 빼면 160만원이다.

항목	값
현재 / 후보 월 이익	150 / 260만원
6개월 이익 증가분	$6 \times (260 - 150) = 660$ 만원
이전 비용 / 보유 현금	500 / 700만원
순편익 / 반환값	160만원 / True

- `agent_rules.py`의 TODO 5 부분을 완성해 보자.
 - `python checks_abm.py --stage 5`
 - 월 이익 150 / 후보 260 / 비용 500 / 현금 700 : True
- 순편익이 0이거나 현금이 비용보다 적으면 이전할까?

- 이전의 이득과 비용을 낼 수 있는지를 모두 확인한다.



```
def should_move_firm(firm, candidate_profit):  
    benefit = 6 * (candidate_profit - firm["profit"]) - firm["move_cost"]  
    return benefit > 0 and firm["cash"] >= firm["move_cost"]
```

실습 6 : 기업 이전 반영

- `move_firm` 함수를 작성한다.
 - 입력 : 행위자 상태 / 기업 ID / 승인된 사업장 ID
 - 이전 사업장을 비우고 새 사업장과 기업의 입주 정보를 맞춘다.
 - 비용을 차감하고 새 사업장의 월 이익을 기업 상태에 저장한다.
 - 반환값 없이 상태를 갱신하며 직원의 고용 관계는 유지한다.

기업 이전 전후의 상태

- F1이 X에서 Y로 이전하면 자금은 200만원, 월 이익은 260만원이 된다.

상태	이전하기 전	이전하고 난 후
F1의 site_id	X	Y
사업장 X / Y	F1 입주 / 빈 사업장	빈 사업장 / F1 입주
cash	700만원	200만원
profit	150만원	260만원

- `agent_rules.py`의 TODO 6 부분을 완성해 보자.
 - `python checks_abm.py --stage 6`
 - F1의 사업장 Y / 현금 200 / 월 이익 260
- 기업이 옮겼다면 직원의 `employer_id`도 바꾸어야 할까?

- 사업장과 기업의 참조를 갱신하고, 이전 비용을 한 번 차감한다.

```
def move_firm(state, firm_id, new_site_id):  
    firm = state["firms"][firm_id]  
  
    state["sites"][firm["site_id"]]["occupant"] = None  
    state["sites"][new_site_id]["occupant"] = firm_id  
  
    firm["site_id"] = new_site_id  
    firm["cash"] -= firm["move_cost"]  
    firm["profit"] = state["sites"][new_site_id]["profit"]
```

- `same_type_ratio` 함수를 작성한다.
 - 이번 함수는 주택 / 기업 도시와 분리된 이웃 격자에서 사용한다.
 - 입력 : 격자 / 현재 행위자의 x 좌표와 y 좌표
 - 상하좌우와 대각선의 최대 8칸에서 같은 유형의 비율을 구한다.
 - 현재 칸과 빈칸을 제외하고, 격자 바깥으로 나가지 않는다.
 - 반환값 : 0~1의 비율 / 이웃이 없으면 None

같은 유형의 이웃 세기

- 가운데 A를 제외한 이웃 중 A는 3명, B는 3명이다.

A	B	
B	A	A
	A	B

- 같은 유형 A 3명
- 다른 유형 B 3명
- 빈칸 2칸
- 같은 유형의 이웃 비율 : $3 / (3 + 3) = 50\%$

- `agent_rules.py`의 TODO 7 부분을 완성해 보자.
 - `python checks_abm.py --stage 7`
 - 이웃 A 3명 / B 3명 / 빈칸 2개 : $3/6 = 0.5$
- 모서리에 있거나 주변 칸이 모두 비어 있다면 어떻게 계산할까?

이웃 비율 해설 : 이웃 검사

- 주변 좌표를 만들고 자신의 칸과 범위를 벗어난 칸을 건너뛴다.

```
def same_type_ratio(grid, x, y):  
    same = 0  
    occupied = 0  
  
    for dy in (-1, 0, 1):  
        for dx in (-1, 0, 1):  
            nx, ny = x + dx, y + dy  
  
            if dx == 0 and dy == 0:  
                continue  
            if not (0 <= ny < len(grid) and 0 <= nx < len(grid[0])):  
                continue
```

이웃 비율 해설 : 비율 반환

- 빈칸을 제외한 이웃 수를 분모로 삼아 같은 유형의 비율을 반환한다.

```
neighbor = grid[ny][nx]

if neighbor is not None:
    occupied += 1

    if neighbor == grid[y][x]:
        same += 1

return same / occupied if occupied else None
```

- 제공 코드는 가구와 기업의 행동을 반영한 뒤 통근과 기록을 갱신한다.

순서	제공 코드의 처리
1	월초 상태로 만족도와 연속 불만족 기간 갱신
2	이주 신청을 모아 주택별로 한 가구 승인
3	승인된 가구 이주 반영
4	분기마다 기업 이전 판단과 반영
5	통근 재계산 / 월말 상태 기록

비교 실험 1 : 기본 도시

- 기본 도시를 연 뒤 0개월의 상태를 확인한다.

확인 항목	초기값
시민 / 가구 / 기업	3명 / 2가구 / 1개
H1의 주택 / 만족도 / 저축	A / 40.5점 / 150만원
F1의 사업장 / 현금	X / 700만원
취업 시민의 평균 통근	30분

비교 실험 2 : 가구 이주

- 1개월씩 진행하여 2개월째 H1의 주택과 저축을 확인한다.

행위자 상태

	0	1	2	3	4
0	R A 0 / 4	R C 0 / 4	R B 2 / 4	R D 1 / 4	물
1	---		---	---	물
2	---	---	---	---	
3	C Y 0 / 8		P 공원	C X 2 / 8	

A~D: 주택 / X~Y: 사업장 / P: 공원

시민	가구	주택	기업	통근(분)
P1	H1	B	F1	20
P2	H1	B	없음	—
P3	H2	D	F1	15

주거지는 소속 가구에서 조회합니다.
통근 시간은 현재 도로와 사업장 위치로 계산합니다.

가구

1개월 진행

6개월 진행

월별 CSV

가구	주택	만족도	저축	연속 불만(월)	대기(월)
H1	B	62.33	120	0	3
H2	D	72.25	100	0	0

조사할 가구

H1

순서대로 최대 3곳 / 개선 폭 10점 이상

후보 주택	만족도	임대료	입주 가능
C	43.33	130	불가
D	70	90	불가
A	40.5	80	가능

기업

기업	사업장	월 이익	이전 자금	고용 시민
F1	X	150	700	P1 / P3

분기마다 이전 검토 / 앞으로의 6개월 이익 비교 / 시민의 고용 관계 유지

2개월: H1 A → B

비교 실험 3 : 기업 이전

- 3개월째 F1의 사업장과 직원의 통근 변화를 확인한다.

행위자 상태

	0	1	2	3	4
0	R A 0 / 4	R C 0 / 4	R B 2 / 4	R D 1 / 4	물
1	---		---	---	물
2	---	---	---	---	
3	C Y 2 / 8		P 공원	C X 0 / 8	

A~D: 주택 / X~Y: 사업장 / P: 공원

시민	가구	주택	기업	통근(분)
P1	H1	B	F1	25
P2	H1	B	없음	—
P3	H2	D	F1	40

주거지는 소속 가구에서 조회합니다.
통근 시간은 현재 도로와 사업장 위치로 계산합니다.

가구

1개월 진행

6개월 진행

월별 CSV

가구	주택	만족도	저축	연속 불만(월)	대기(월)
H1	B	58.17	120	0	2
H2	D	51.42	100	0	0

조사할 가구

H1

순서대로 최대 3곳 / 개선 폭 10점 이상

후보 주택	만족도	임대료	입주 가능
C	60	130	불가
D	49.17	90	불가
A	73.83	80	가능

기업

기업	사업장	월 이익	이전 자금	고용 시민
F1	Y	260	200	P1 / P3

분기마다 이전 검토 / 앞으로의 6개월 이익 비교 / 시민의 고용 관계 유지

2개월: H1 A → B
3개월: F1 X → Y

기업 이전과 직원의 통근

- 기업의 이익이 늘어나도 직원의 통근 시간이 짧아지는 것은 아니다.

항목	2개월 / X	3개월 / Y
F1의 월 이익	150만원	260만원
P1의 통근	20분	25분
P3의 통근	15분	40분
취업 시민 평균	17.5분	32.5분

이웃 관계 실험의 조건

- 이웃과 도시 변화 탭은 이웃 구성만으로 이동을 결정하는 별도 격자다.

조건	설정
격자 / 인원	8 × 8 / A 25명 / B 25명
빈칸 / 난수 시드	14개 / 42
이웃 범위	상하좌우와 대각선의 최대 8칸
만족 기준	같은 유형의 이웃 비율 50% 이상

비교 실험 4 : 이웃의 분포

- 만족 기준 50%에서 초기 배치와 1회차 뒤의 배치를 비교한다.

초기 배치 / 불만족 21명

A	B	A	A	B	B	A	
B	A	B	A	B	B	A	A
B	B	B	A	A		B	B
A	B	A		A			B
A	A		B	B	B		A
B		B	B		A		
A	B	B	A	B		B	A
	A		A	A	B	A	A

1회차 / 불만족 2명

B	B	A	A	B	B	A	A
B		A	A		B		A
B	B		A	A	A	B	B
A		B		A	A		B
A	A		B	B		A	A
B	B	B	B	B		A	A
B	B	B			A		A
B	B		A	A	B	A	A

비교 실험 5 : 만족 기준

- 같은 초기 배치에서 만족 기준을 25% / 50% / 75%로 바꾸어 비교한다.

고정할 조건	바꿀 조건	기록할 결과
격자 / 인원 / 빈칸 / 시드 42	만족 기준	불만족 수 / 이동 횟수 / 분포

직접 해보기 : 저장한 도시 복원

가상공간과 메타버스
Agent-based City Modeling Lab

- 3개월의 도시를 저장한 뒤 불러와 보자.
 - 상태 저장으로 JSON을 저장하고, 기본 도시로 초기화한다.
 - 불러오기 후 H1의 주택 B / F1의 사업장 Y를 확인한다.
- 월별 CSV와 JSON에는 각각 어떤 자료가 들어 있을까?

- 통근 / 주거비 / 환경으로 가구의 만족도를 계산했다.
- 기간과 자원을 확인하고 조건을 충족하는 주택을 선택했다.
- 가구와 기업의 이동에 맞추어 입주 관계와 자금을 갱신했다.
- 개별 행동이 통근과 이웃 분포에 미치는 영향을 비교했다.

감사합니다.

utilForever@gmail.com

<https://github.com/utilForever>

Facebook, Twitter, Instagram, LinkedIn: utilForever